

Projet FoSyMa 2025–2026

Patrouille Multi-agent dans Dedale

Exploration coopérative et capture coordonnée de Golems

Gaya LAHOUAZI — 21308739

Encadrant : Cédric Herpson

Département d'Informatique, Faculté des Sciences et Ingénierie
Sorbonne Université
Master Informatique — AI2D

Version : 30 avril 2026

Table des matières

1 Introduction 3
2 Architecture générale et diagrammes UML 4
2.1 Structure du projet 4
2.2 Diagramme UML — Agents et comportements 4
2.3 Diagramme UML — Module knowledge 5
2.4 Diagramme de séquence — protocole de chasse 6
3 Choix de conception 7
3.1 Exploration 7
3.2 Communication 7
3.3 Coordination 8
3.4 Collecte 9
4 Analyse des algorithmes 9
4.1 Exploration coordonnée par priorité lexicographique 9
4.2 WumpusTracker — triangulation Bayésienne 10
4.3 Stratégie de blocage greedy 10
5 Conclusion 11
Références 11

1 Introduction

Le projet **FoSyMa 2025–2026** de l'UE *Fondements des Systèmes Multi-Agents* (Master Informatique AI2D) propose de développer, sur la plateforme JADE et l'environnement Dedale, une variante multi-agent du jeu *Hunt the Wumpus* (G. Yob, 1972). Une équipe d'agents coopératifs doit **explorer un environnement inconnu** représenté par un graphe non-orienté, puis **chasser et bloquer** un ou plusieurs Golems présents sur la carte.

Le projet présente trois défis multi-agents classiques :

- **Coordination décentralisée** sans coordinateur central, avec des communications limitées en portée et hétérogènes entre agents ;
- **Inférence partielle** : la position du Golem n'est pas directement observable. Sa présence n'est trahie que par les odeurs (*Stench*) qu'il diffuse dans son voisinage ;
- **Garantie de terminaison** dans un environnement dynamique : le Golem se déplace pendant la chasse.

Périmètre traité. Le rapport couvre l'**exploration coopérative** (section 2 du sujet) et la **patrouille / capture** (section 3 du sujet, scénario 1 Golem sur arbres et graphes ; scénario multi-Golems en best-effort). La **collecte de trésors** n'est pas implémentée dans ce périmètre — son intégration future est discutée en §3.4. La **distribution multi-machines** (semaine 9) est explicitement hors périmètre.

Plan. La section 2 décrit l'architecture globale du code et présente trois diagrammes UML (classes agents et behaviours, classes *knowledge*, séquence d'un tick de chasse). La section 3 motive les choix de conception sur les quatre axes *exploration, communication, coordination, collecte*. La section 4 analyse chaque algorithme : forces, limites, complexité (temps / mémoire / communication), optimalité et critère d'arrêt. La section 5 propose une synthèse, un regard critique et des pistes d'extension.

2 Architecture générale et diagrammes UML

2.1 Structure du projet

Le code utilisateur est organisé en trois packages Java sous `eu.su.mas.dedaleEtu.mas`:

Package	Rôle	Fichiers ajoutés
agents	Cycle de vie, enregistrement DF, état partagé	MultiExploreAgent, HuntAgent
behaviours	Boucles de décision et dispatchers ACL	MultiExploreBehaviour, HuntBehaviour, ReceiveShareBehaviour, ReceiveHuntBehaviour
knowledge	Modèles de données et inférence	MapRepresentation (étendue), WumpusTracker, HuntSightingMessage

Trois fichiers JSON de configuration sont fournis dans `resources/` : `agent-2hunt.json` (2 chasseurs + 1 Golem, R=1), `agent-3hunt.json` (3 chasseurs + 1 Golem, R=2), `agent-3hunt-2golems.json` (3 chasseurs + 2 Golems).

2.2 Diagramme UML — Agents et comportements

La hiérarchie d'héritage suit fidèlement l'API de JADE et de Dedale. Les éléments abstraits hérités (en gris) ne sont pas modifiés. `HuntAgent` étend `MultiExploreAgent` via une méthode patron `createBehaviours()`, ce qui permet de réutiliser tout le code de coordination (DF, intentions, vote DONE) sans duplication.

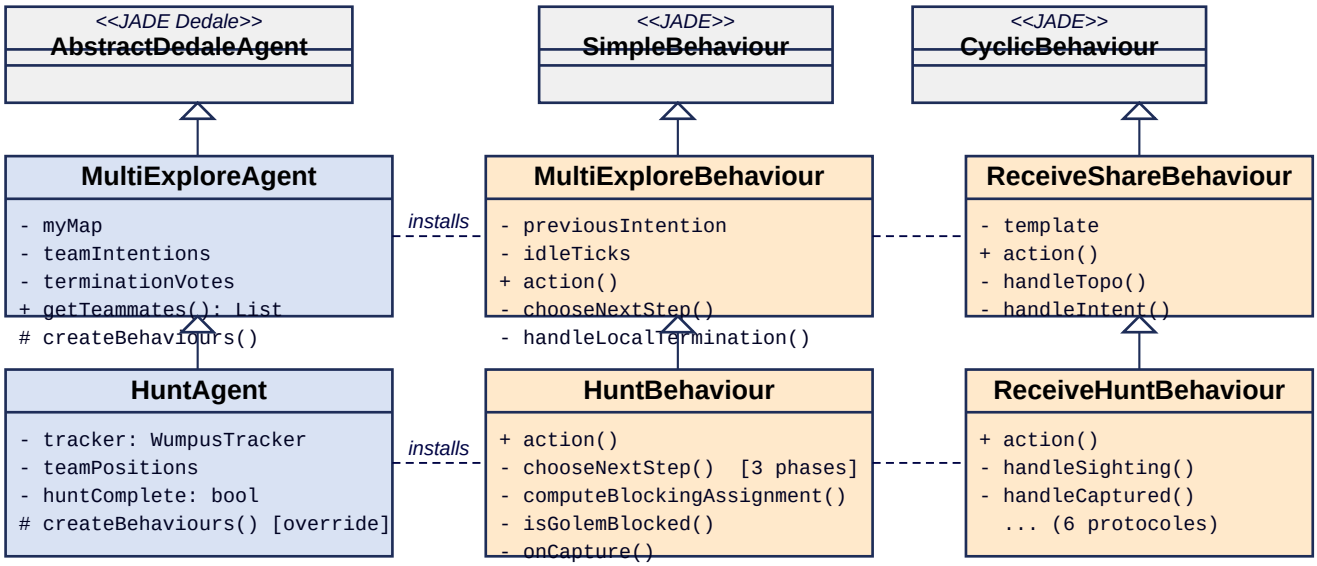


***Figure** : diagramme de classes — agents (bleu) et comportements (orange). Les flèches à pointe creuse indiquent l'héritage, les pointillés une dépendance « installs ».*

2.3 Diagramme UML — Module knowledge

Les classes du module `knowledge` sont indépendantes de JADE : elles encapsulent respectivement la topologie partielle (`MapRepresentation`), l'inférence sur la position du Golem (`WumpusTracker`) et le payload sérialisable des messages de sighting. Cette séparation permet de tester le tracker en isolation, sans démarrer la plateforme.

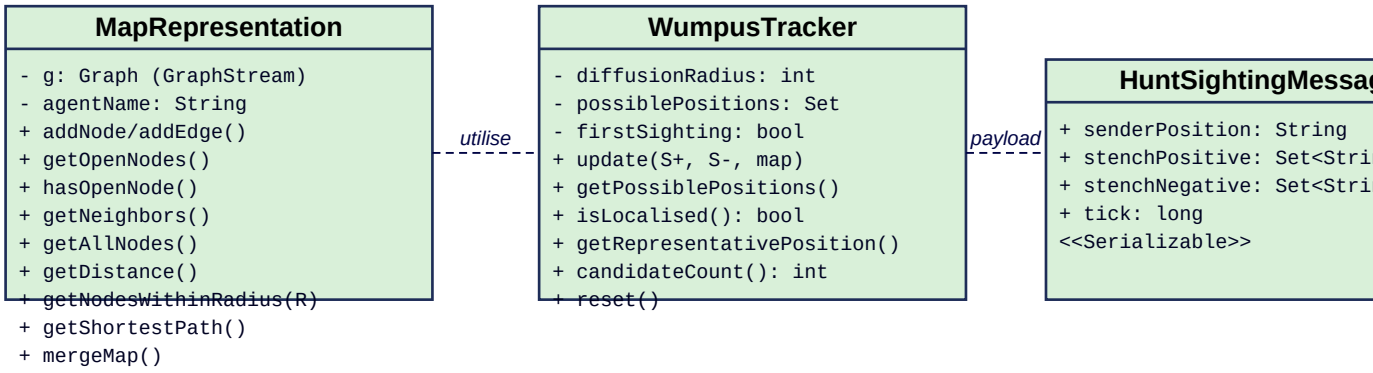


***Figure** : diagramme de classes — module knowledge.*

2.4 Diagramme de séquence — protocole de chasse

Le diagramme ci-après illustre une itération typique côté `HuntBehaviour` : observation locale, mise à jour du tracker, broadcast aux coéquipiers, choix du prochain pas et action. La vérification de capture est faite à la fin du tick suivant l'observation, ce qui garantit une réponse en au plus 1 tick après immobilisation effective du Golem.

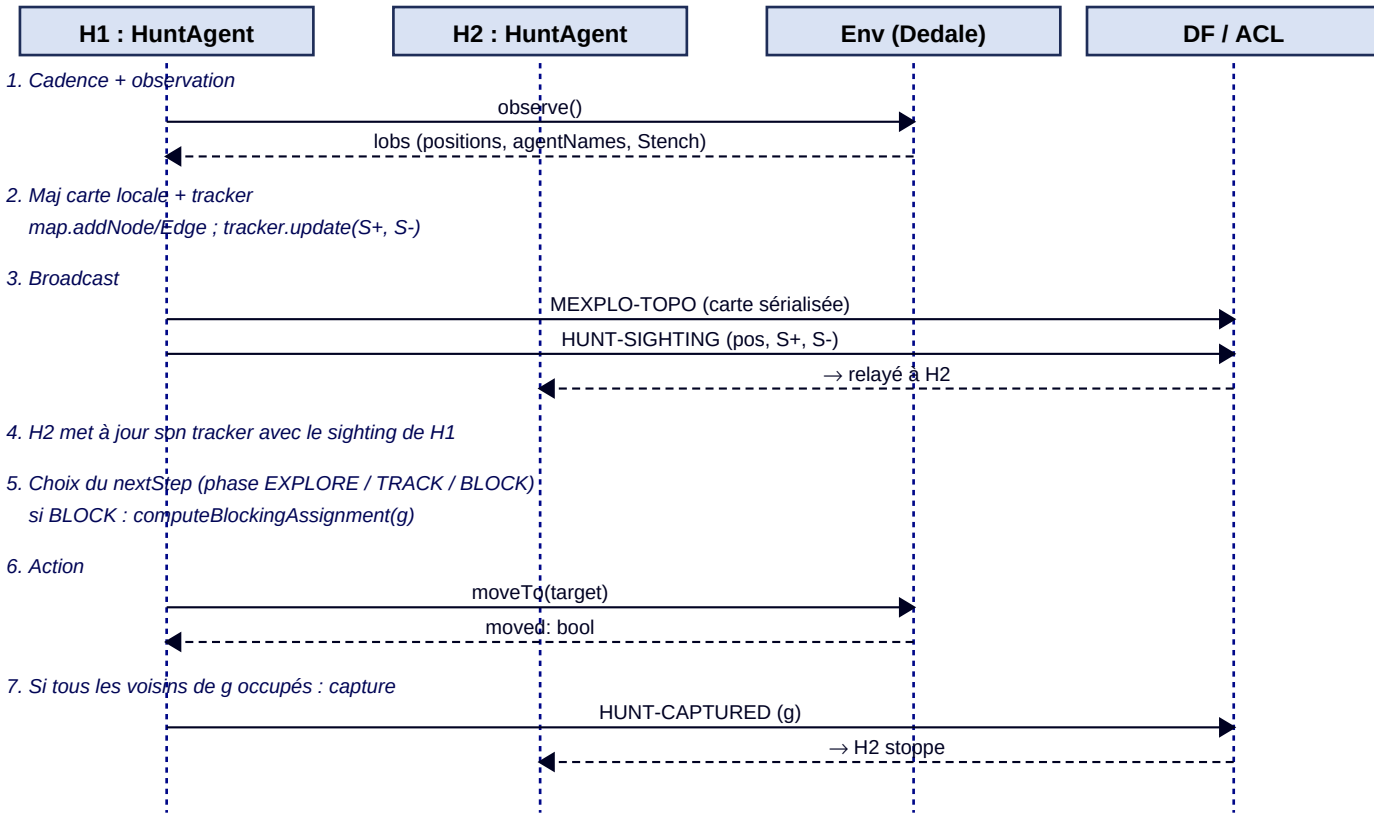


***Figure** : séquence simplifiée d'un tick de chasse entre H1 et H2.*

3 Choix de conception

3.1 Exploration

L'algorithme de base est une **DFS coordonnée**. Chaque agent ferme son nœud courant (passage à l'attribut `closed`) et marque ses voisins non visités comme `open`. Pour éviter la convergence en file unique observée dans `ExploreCoopAgent`, chaque agent annonce son nœud cible (`MEXPLO-INTENT`) et applique une règle d'évitement :

Soit i un agent et T(j) la cible déclarée d'un coéquipier j. L'agent i évite tout T(j) tel que name(j) < name(i) lexicographiquement.

Cette règle est **déterministe** (deux agents qui partagent la même information calculent la même décision), **asymétrique** (l'agent de plus petit nom prend la priorité) et **locale** (aucun protocole de consensus n'est nécessaire). Le choix du prochain pas se fait en trois passes : (1) voisin direct ouvert non-évité, (2) voisin direct ouvert (avoidance ignorée — anti-deadlock), (3) Dijkstra vers le nœud ouvert le plus proche parmi les non-évités.

3.2 Communication

Découverte par DF. Au `setup()`, chaque agent s'enregistre dans le Directory Facilitator de JADE sous le service `"explo-team"`. La méthode `getTeammates()` interroge le DF avec un cache TTL de 5 s, ce qui évite un lookup à chaque tick tout en restant réactif aux nouveaux arrivants. Aucun nom de coéquipier n'est codé en dur.

Six protocoles ACL, tous en performatif INFORM :

Protocole	Contenu	Cadence	Phase

MEXPLO-TOPO	SerializableSimpleGraph (carte)	Chaque tick	Toujours
MEXPLO-INTENT	String — id du nœud cible	Sur changement	Toujours
MEXPLO-DONE	(vide)	Au passage à terminé + toutes 10 idle	Exploration
MEXPLO-UNDONE	(vide)	Sur rétractation	Exploration
HUNT-SIGHTING	HuntSightingMessage (pos, S+, S–)	Chaque tick	Chasse
HUNT-CAPTURED	String — nœud de blocage	Sur détection	Chasse

Réception centralisée par agent. Un unique `CyclicBehaviour` (`ReceiveShareBehaviour` ou `ReceiveHuntBehaviour`) filtre tous les protocoles via une `MessageTemplate.or(...)` pré-construite et dispatch sur les handlers internes. Cela évite la prolifération de *n* CyclicBehaviour concurrents et limite le coût de filtrage à un parcours unique.

3.3 Coordination

Trois mécanismes complémentaires sont utilisés selon la phase :

- **Évitement par priorité lex** (exploration) : décrit en §3.1. Asymétrie déterministe sans consensus.
- **Vote 2-phases pour la terminaison** (exploration) : un agent émet `DONE` dès que sa carte n'a plus de nœud ouvert, et rétracte (`UNDONE`) si une carte reçue rouvre des nœuds. La terminaison globale est confirmée lorsque tous les coéquipiers ont voté.
- **Assignation greedy déterministe** (chasse) : quand le Golem est localisé, chaque agent calcule la même assignation chasseur→voisin par double tri lexicographique (chasseurs ordonnés, voisins ordonnés). Le coordinateur est *implicite* et réplicable.

L'absence de leader / coordinateur central est volontaire : elle simplifie drastiquement le code, supprime le risque de point de défaillance unique, et rend l'algorithme robuste à l'arrivée ou au départ d'un agent (DF dynamique).

3.4 Collecte (non implémentée — discussion)

La collecte de ressources (or, diamant) n'est pas requise pour les sections du sujet traitées dans ce projet. Elle est néanmoins présente dans l'API Dedale (`pick()`, `openLock()`, `emptyMyBackPack()`) et exemplifiée par les agents fournis `DummyCollectorAgent` et `DummyTankerAgent`. Si on voulait l'intégrer à l'architecture proposée :

- Étendre `MapRepresentation` avec un attribut *treasure* par nœud (id, type, quantité) — la sérialisation existante via `SerializableSimpleGraph` supporte déjà des attributs génériques ;
- Ajouter un protocole `MEXPLO-LOOT` qui propage les découvertes de trésors avec leur position, type et quantité ;
- Mettre en place un *contract-net protocol* entre collecteurs et tanker pour négocier les rendez-vous d'évacuation (`emptyMyBackPack`) ;
- Étendre l'observable `WIND` (associé aux puits) à un module de tracking similaire à `WumpusTracker` pour cartographier les zones dangereuses.

L'ajout serait orthogonal aux mécanismes décrits jusqu'ici : un nouveau `CollectAgent` hériterait de `MultiExploreAgent` via le hook `createBehaviours()`, et un `HuntCollectAgent` combinerait les deux.

4 Analyse des algorithmes

4.1 Exploration coordonnée par priorité lexicographique

Description. Chaque agent maintient une `MapRepresentation` ; à chaque tick, il ferme son nœud, ouvre les voisins inconnus, broadcast la carte et choisit le prochain pas selon la stratégie en 3 passes (§3.1). La règle d'avoidance lex assure une répartition implicite du travail.

Forces

- Décentralisé, sans leader ni protocole de consensus ;
- Robuste à l'arrivée/départ d'un agent (DF dynamique) et aux pertes de message (décision basée sur l'information localement disponible) ;
- Repli en cas d'avoidance vide : pas d'interblocage statique (passe 2).

Limites

- Diffusion *full-map* à chaque tick : la taille du message croît linéairement avec |V|, ce qui pénalise les très grands graphes (à optimiser par delta-share) ;
- La priorité lex n'est **pas équitable** : l'agent de plus grand nom fait systématiquement les trajets « second choix » ;
- Sur deux composantes connexes disjointes, le vote DONE ne converge jamais (hypothèse de connexité du sujet).

Complexité

Ressource	Coût par tick / par agent	Coût total
Temps	O(d) observation + O(|V| log|V|) Dijkstra	O(|V|² log|V|) jusqu'à carte complète
Mémoire	O(|V| + |E|) carte + O(|H|) intentions	O(|V| + |E|) par agent
Communication	O(|H| · |V|) octets par tick (broadcast carte)	O(|V|² · |H|) jusqu'à terminaison

Optimalité. L'algorithme *n'est pas optimal* : un partitionnement explicite (par exemple Voronoi sur les positions initiales) donnerait moins de visites en doublon. Cependant la priorité lex offre un bon compromis simplicité / efficacité, sans négociation préalable. **Critère d'arrêt** : l'agent vote DONE quand `!myMap.hasOpenNode()` ; la terminaison globale est confirmée quand `terminationVotes` contient tous les coéquipiers (vote 2-phases avec rétractation).

4.2 WumpusTracker — triangulation Bayésienne

Description. Maintien d'un ensemble P de positions possibles du Golem. À chaque mise à jour, on étend P d'un saut (modèle de mouvement), puis on intersecte avec les boules de rayon R autour des nœuds stench-positifs et on soustrait celles autour des stench-négatifs.

$$P \leftarrow (P \cup N(P)) \cap \blacksquare_{x \in S^+} B(x, R) \setminus \blacksquare_{y \in S^-} B(y, R)$$

Forces

- **Cohérence** : la position réelle reste toujours dans P (preuve par récurrence sur l'invariant : intersection et différence préservent l'appartenance) ;
- **Décroissance monotone** à information fixe : $|P|$ ne grandit jamais entre deux observations consécutives ;
- Modulaire et testable en isolation (aucune dépendance JADE).

Limites

- Suppose **1 seul Golem** : avec plusieurs Golems, S^+ grandit, l'intersection devient vide et le tracker se réinitialise — solution proposée : clusterisation spatiale, `MultiWumpusTracker` ;
- Le rayon *R* doit être connu (paramètre `stenchRadius`) ; une mauvaise estimation conduit à des incohérences ;
- Si le Golem disparaît du voisinage observé, l'expansion par 1 saut peut faire grossir *P* à chaque tick : à long terme |P|→|V| (perte de localisation).

Complexité

Ressource	Coût par update
Temps	$O(\|S^{\pm}\| \cdot d^R)$ (BFS rayon R)
Mémoire	$O(\|P\|) \leq O(\|V\|)$
Communication	Chaque sighting : $O(\|S^{\pm}\|)$ octets, broadcast à $O(\|H\|)$

Optimalité. Le tracker est *optimal au sens de l'information* : il calcule exactement l'ensemble des positions compatibles avec les observations cumulées, sans approximation. Une approche probabiliste (filtre Bayésien à distribution) donnerait une information plus fine mais nécessiterait une connaissance du modèle de mouvement du Golem (non disponible dans Dedale). **Critère d'arrêt** de l'inférence : `isLocalised()` retourne vrai dès que $|P| = 1$; le tracker est ensuite figé jusqu'au prochain mouvement.

4.3 Stratégie de blocage greedy

Description. Quand le Golem est localisé en *g*, on calcule l'assignation chasseur→voisin par double tri lex (déterministe), puis chaque agent va vers son voisin via Dijkstra. La capture est confirmée dès que tous les voisins de *g* sont occupés par des chasseurs (les positions sont connues via le piggyback `senderPosition` des sightings).

Forces

- **Déterministe** : tous les chasseurs calculent la même assignation à partir de la même information — pas de protocole de validation à 2 phases ;
- Pas de point de défaillance : si un agent crashe, les autres continuent (DF retire automatiquement, le cache TTL le rafraîchit en ≤ 5 s) ;
- Simple à raisonner et à étendre.

Limites

- **Non-optimale** : une affectation par algorithme hongrois (matching de coût minimal) donnerait une somme totale de distances optimale ;
- **Sous-effectif** : si $|H| < \deg(g)$, le Golem peut s'échapper indéfiniment ; sur un arbre, une stratégie *push-to-leaf* (non implémentée) ramène le besoin à 1 chasseur ;
- Pas de mécanisme de réassignation rapide si un chasseur est bloqué (collision répétée) : il finira par changer de cible au tick suivant via le `previousIntention=null`, mais ça ajoute des allers-retours.

Complexité

Ressource	Coût par tick	Coût jusqu'à capture
Temps	O(|H| · |V| log|V|) (Dijkstra par paire)	O(diam(G) · |V|² · |H|)
Mémoire	O(|H|) assignation + O(|H|) positions	O(|V| + |H|)
Communication	O(|H|) — sightings continuent	O(diam(G) · |H|²)

Optimalité. Greedy lex est *déterministe* et *cohérent*, mais pas optimal au sens du coût total. Sur le critère du **nombre minimal de chasseurs**, la version implémentée requiert deg(*g*) ; la version optimale connue (push-to-leaf sur arbre) ramène à 1. **Critère d'arrêt** de la chasse : `isGolemBlocked(g)` renvoie vrai (tous les voisins de *g* sont des positions de chasseurs), un `HUNT-CAPTURED` est diffusé, et chaque agent vérifie `isHuntComplete()` au tick suivant pour terminer son comportement.

5 Conclusion

5.1 Synthèse

Le projet propose une architecture unifiée pour les deux étapes du sujet : un agent `HuntAgent` qui hérite d'un `MultiExploreAgent` robuste, et y ajoute l'inférence Bayésienne sur les odeurs ainsi qu'une stratégie d'assignation déterministe pour la capture. La modularité du code (séparation tracker / strategy / protocol / map) facilite la maintenance et permet de tester chaque composant en isolation. Les trois algorithmes principaux (exploration coordonnée, triangulation, blocage greedy) ont chacun des garanties théoriques explicites et des complexités documentées en Javadoc et en §4.

5.2 Regard critique

- **Diffusion full-map** à chaque tick : viable sur les topologies d'examen mais scaling-limité ; un mécanisme de delta-share aurait été préférable dès le début ;
- **Tracker mono-Golem** : choix volontaire pour valider l'inférence sur le cas simple, mais limite la robustesse multi-Golems. La voie vers `MultiWumpusTracker` est claire mais demande encore du temps de développement ;
- **Stratégie de blocage non-optimale** : sur arbres, l'encerclement gaspille des chasseurs là où push-to-leaf serait optimal. La preuve d'optimalité (1 chasseur suffit sur un arbre fini) est connue ; l'implémentation est délicate (calcul d'anti-chemin dynamique en présence d'un Golem qui bouge) ;
- **Tests automatisés absents** : la validation s'est faite à la console, sur un petit nombre de scénarios. Une suite JUnit sur `WumpusTracker` et `MapRepresentation` aurait sécurisé les régressions ;
- **Cache DF à TTL fixe** : un mécanisme d'invalidation événementielle (sur réception d'un message d'un nouvel agent) serait plus réactif.

5.3 Extensions et améliorations possibles

- **MultiWumpusTracker** : clusterisation spatiale des nœuds stench-positifs en composantes connexes (avec arêtes ≤ 2R), un tracker par cluster ; assignation des chasseurs par composante ;
- **Push-to-leaf sur arbres** : algorithme dédié qui réduit le besoin à 1 chasseur ; calcul d'un anti-chemin qui force le Golem vers la feuille la plus proche ;
- **Delta-share** pour `MEXPLO-TOPO` : ne transmettre que les nœuds/arêtes ajoutés depuis le dernier broadcast à chaque destinataire (mémoire locale par paire) ;
- **Algorithme hongrois** pour l'assignation : remplace le greedy par un matching de coût minimal (toujours déterministe) ;

- **Ajout de la collecte** : `CollectAgent` avec protocole `MEXPLO-LOOT` et coordination par contract-net avec un tanker ;
- **Distribution multi-machines** : configuration `PLATFORMisDISTRIBUTED=true` et répartition des agents par `agentInitialContainer`. L'architecture DF + ACL est nativement compatible.

Références

[1] C. Herpson, *Sujet de Projet FoSyMa 2025–2026 — Patrouille Multi-agent*, M1 AI2D, Sorbonne Université, 2026.

[2] C. Herpson, *TP n°2 — Projet Dedale : Exploration de l'Environnement*, M1 AI2D, Sorbonne Université, 2026.

[3] G. Yob, “Hunt the Wumpus”, *Creative Computing*, vol. 1, n° 5, 1975, p. 51–54.

[4] FIPA, *FIPA ACL Message Structure Specification*, document SC00061G, 2002.

[5] F. Bellifemine, G. Caire, D. Greenwood, *Developing Multi-Agent Systems with JADE*, Wiley Series in Agent Technology, 2007.

[6] *Dedale Documentation*, https://dedale.gitlab.io.

[7] *GraphStream Project — A Dynamic Graph Library*, http://graphstream-project.org/.

[8] S. Russell et P. Norvig, *Artificial Intelligence : A Modern Approach*, 3e éd., Prentice Hall, 2010.